

Scrubless - Architecture

Semantic video search · *Stop scrubbing. Start searching.*

Ernest Opara

2026-05-22

Contents

1 Overview	2
1.1 Design principles	2
2 Technology stack	4
3 How it works	5
3.1 Indexing pipeline	5
3.2 Search pipeline	5
4 Data model	7
5 API reference	8
6 Accounts, tiers & billing	9
6.1 Configuration (environment variables)	9
7 Deployment	10
7.1 Project layout	10
8 Security model	12
9 Admin dashboard	13
10 Roadmap - V2 (weekend of 2026-05-23)	14
11 Roadmap - V3	16
12 Known limitations (V1)	17
13 About this document	18
14 Change log	18

1 Overview

Scrubless is a semantic video-search product. The whole user story is one sentence:

Upload a video → index it → search it in natural language → click a result and the player jumps to that exact moment.

Every uploaded video is split into still frames and an audio transcript. Each frame becomes a **CLIP embedding** - a vector capturing what the frame *looks like* - stored in a vector database alongside the transcript text for that moment. A search query is embedded the same way, and the database returns the frames whose meaning is closest to the query.

The governing constraint (from CLAUDE.md) is **radical simplicity**: one language (Python), one backend file (`app.py`), one frontend file (`index.html`), no Docker-for-dev, no microservices, no build step.

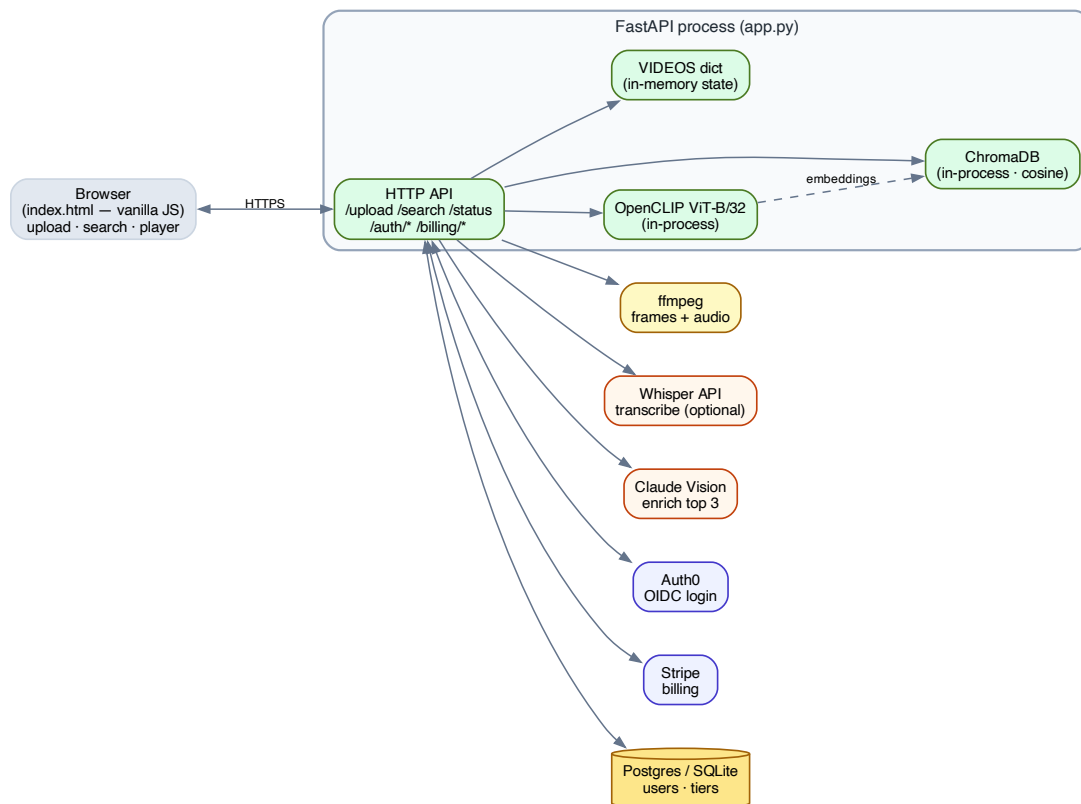


Figure 1: System architecture - one FastAPI process with the model and vector DB in-process; everything else is an external call.

1.1 Design principles

- **One file per layer.** All backend logic in `app.py`; all UI in `index.html`.

- **In-process everything.** CLIP and the vector DB run inside the FastAPI process - no sidecars, no network hops.
- **Graceful degradation.** Every external dependency (Whisper, Claude, Auth0, Stripe, Postgres) is optional; a missing key disables that feature instead of crashing the app.
- **Cheap to run.** Fits on one small Railway service / a single VPS.

2 Technology stack

Concern	Choice
Web framework	FastAPI - 14 sibling modules registered as routers, <code>app.py</code> is the thin entry
ASGI server	uvicorn
Video processing	ffmpeg via subprocess (frames + audio)
Visual embeddings	OpenCLIP ViT-B/32 (laion2b_s34b_b79k)
Vector search	ChromaDB (in-process <code>PersistentClient</code> , cosine)
Transcription	OpenAI Whisper API (<code>whisper-1</code>) - optional
LLM (Vision / chapters / Q&A)	Anthropic Claude (<code>claude-sonnet-4-6</code>)
Rate limiting	<code>slowapi</code> (per-IP, in-memory)
Frontend	One <code>index.html</code> - vanilla JS + CSS, no build step
Auth	Auth0 (OIDC) via Authlib + signed session cookies
Accounts DB	SQLAlchemy 2.0 - SQLite (dev) / Postgres (prod)
Billing	Stripe Checkout + Customer Portal + signed webhooks
Deployment	Docker image on Railway; custom domain via GoDaddy
Docs	Markdown + Graphviz, rendered to PDF by pandoc
PowerPoint export	<code>python-pptx</code> (pixel-match each PDF page → 16:9 slide)

Deliberately *not* used: Go, Docker Compose for dev, Terraform/AWS/ECS/S3, React build pipeline, face recognition, knowledge graphs, YouTube video downloads, clip export (deferred or rejected per `CLAUDE.md`).

3 How it works

The FastAPI process is the entire backend. The only stateful stores are **ChromaDB** on disk (frame embeddings + metadata), the **relational DB** (the users table), and the in-memory **VIDEOS** dict (per-video processing status, lost on restart - acceptable for V1).

3.1 Indexing pipeline

When a video is uploaded, `process_video()` runs in a background thread.

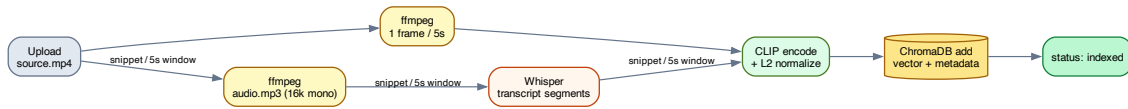


Figure 2: Indexing - frames and audio are extracted in parallel, joined per 5-second window, embedded, and written to ChromaDB.

1. **Frames** - ffmpeg at $\text{fps}=1/5$: one JPEG every **5 s** (`FRAME_INTERVAL`).
2. **Audio** - ffmpeg to 16 kHz mono MP3, small enough for Whisper. Wrapped in `try/except`: a video with no audio indexes visually and continues.
3. **Transcript** - Whisper(verbose_json) $\rightarrow$ `[{start, end, text}]`. Skipped silently without `OPENAI_API_KEY`.
4. **Embed** - each frame is CLIP-encoded, L2-normalized, and paired with the transcript overlapping its 5-second window.
5. **Store** - one `segments.add()` call: `id = "{video_id}-{i}"`, the vector, metadata `{video_id, timestamp, frame_path, transcript_segment}`.
6. **Status** - progress updates per frame; `status` $\rightarrow$ `indexed` (or error).
7. **Chapters (V3)** - if a transcript exists, one Claude call (`summarize_video`) produces a 2–3 sentence summary + 4–8 timestamped chapters, stored on the `videos` row (best-effort).

3.2 Search pipeline

`POST /api/search/{video_id}` embeds the query into the same CLIP space and asks ChromaDB for the nearest frames.



Figure 3: Search - the query is embedded, matched against frame vectors, and the top 3 are enriched with a Claude Vision description.

1. Reject if the video isn't indexed (409) or the query is empty (400).

2. **Embed** the query (embed_text).
3. **Query** ChromaDB for the top **10**, filtered to the one video_id, ranked by cosine distance.
4. Convert distance to a **score** ($1.0 - \text{dist}$); attach frame URL, timestamp, transcript snippet.
5. **Enrich** the top **3** (ENRICH_TOP_N) with a 1–2 sentence Claude Vision description (best-effort; skipped without an Anthropic key).
6. Return the list; the UI renders clickable thumbnails that seek the player.

4 Data model

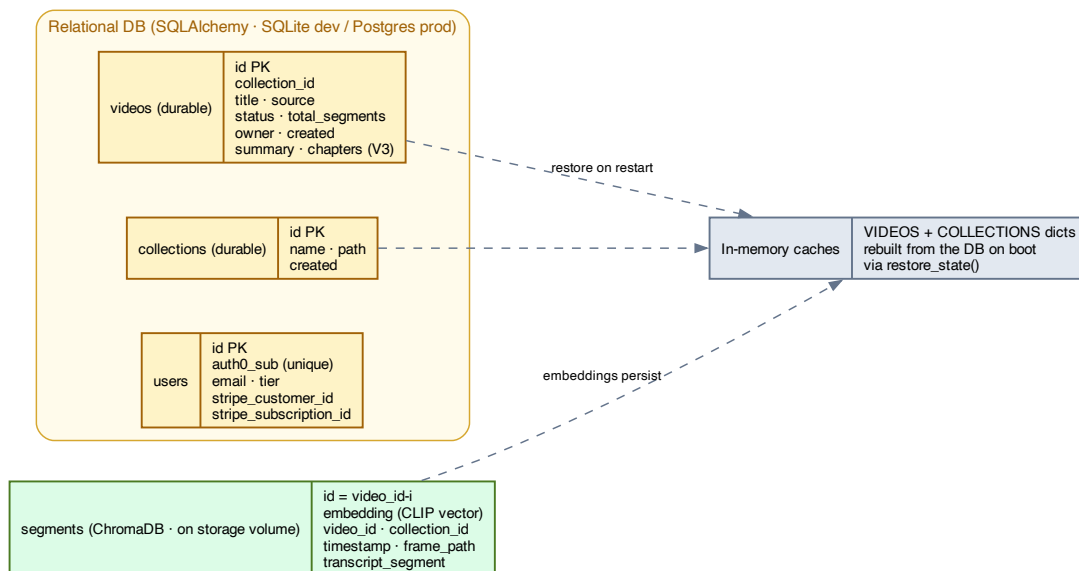


Figure 4: The relational DB holds durable videos + collections alongside users; ChromaDB holds the embeddings on the storage volume; the in-memory dicts are caches rebuilt on boot.

- **users / videos / collections** (SQLAlchemy) - the relational store. `users` holds accounts/tiers/Stripe IDs; `videos` and `collections` (V2) hold the **durable** metadata (title, source, status, owner, membership, plus the V3 summary + chapters) so the library survives a restart. `DATABASE_URL` picks the backend: SQLite in dev, Postgres in prod (`postgres://` normalized to `postgresql://`). New columns are added by an idempotent `ensure_columns()` migration at startup.
- **segments** (ChromaDB, on the storage volume) - one row per frame: embedding + metadata (`video_id`, `collection_id`, `timestamp`, `frame_path`, `transcript_segment`). Distance space: **cosine**.
- **events** (relational) - (`id`, `kind`, `at`). One row per page view / search / Q&A / upload / reel, written fire-and-forget by `record_event()`. Feeds the admin dashboard's totals, 24h / 7d windows, and per-day sparkline.
- **VIDEOS / COLLECTIONS / REELS** (in-memory, in `state.py`) - fast caches of per-video status, folder membership, and reel-build status, rebuilt from the DB on boot by `restore_state()` (any video left mid-index is resumed). Anonymous ownership is tracked in the signed session cookie under `my_videos` / `my_collections` / `my_reels`, capped at 200 entries each.

5 API reference

Every {id} route resolves through the ownership gate (require_video / require_collection / can_access_reel): a 404 is returned to anyone who doesn't own the resource (admins see everything; the sample video is public). Rate limits are per-IP and shown as the [N/window] suffix where set.

Method	Path	Purpose
GET	/	Serve index.html
GET	/admin	Serve index.html (SPA shows admin view if admin)
GET	/storage/{path}	Gated stream of any file under STORAGE (frames, source.mp4, reels) - replaces the previous public static mount
-	/scrubby/*	Static mount (mascot SVGs only - no user content)
POST	/api/upload	Upload a video, start indexing - cap-gated, [20/h]
GET	/api/status/{id}	Poll indexing status / progress
POST	/api/search/{id}	Natural-language search of one video - [60/min]
POST	/api/qa/{id}	Ask a question about a video, cited (V3) - [20/min]
DELETE	/api/videos/{id}	Delete a video (sample is protected)
GET	/api/videos/{id}/source	Stream a video file, range-seekable (V2)
POST	/api/library/pick	Native folder chooser - localhost-only (V2)
POST	/api/library/scan	Index every video under a folder - localhost-only (V2)
POST	/api/library/create	Create an empty collection (upload) (V2)
POST	/api/library/{id}/upload	Upload a video into a collection (V2) - [60/h]
GET	/api/library/{id}	Collection status + per-video progress (V2)
POST	/api/library/{id}/search	Search across a whole collection (V2) - [60/min]
POST	/api/library/{id}/qa	Ask across a collection, cited (V3) - [20/min]
POST	/api/reel	Build a highlight reel from moments (V3) - [10/h]
GET	/api/reel/{id}	Reel build status + URL (V3)
GET	/api/auth/login	Redirect to Auth0 Universal Login - [20/min]
GET	/api/auth/callback	OIDC callback → create/find user, session
GET	/api/auth/logout	Clear session + Auth0 logout
GET	/api/auth/me	Current user (incl. is_admin), upload cap, flags
GET	/api/me/library	Signed-in user's own videos + folders (V2)
POST	/api/billing/checkout	Create a Stripe Checkout session
POST	/api/billing/portal	Open the Stripe Customer Portal
POST	/api/billing/webhook	Stripe webhook → update tier (signature-verified)
GET	/api/admin/stats	Operator dashboard JSON - gated by ADMIN_EMAILS

6 Accounts, tiers & billing

Access is gated by upload size; anonymous visitors are treated as the free tier.

Tier	Upload cap	Price	Account required
Free	500 MB	\$0	No (anonymous)
Pro	2 GB	\$10 / mo	Yes
Studio	10 GB	\$30 / mo	Yes

- **Caps** are enforced twice in `/api/upload`: a fast Content-Length reject, then a streaming backstop that deletes the partial file if the body overruns.
- **Free-tier hygiene** - anonymous uploads auto-expire after **24 h** (`expiry_sweep`, every 30 min). The sample and account-owned videos never expire.
- **Login** is Auth0 OIDC via Authlib; the session is a signed cookie. New subjects create a User row on first callback.
- **Billing** is Stripe hosted Checkout (subscription mode). Tier changes are driven by signature-verified `customer.subscription.*` webhooks, keyed by Stripe customer id. The Customer Portal handles upgrades/cancellation.

6.1 Configuration (environment variables)

Variable	Enables / controls
OPENAI_API_KEY	Whisper transcription (else visual-only)
ANTHROPIC_API_KEY	Claude Vision enrichment + chapters + Q&A
SESSION_SECRET / AUTH0_SECRET	Signed session-cookie key - must be ≥32 chars on HTTPS or the app refuses to boot
APP_BASE_URL	Public base URL for OAuth/Stripe redirects; <code>https://</code> flips on Secure cookies + HSTS
AUTH0_DOMAIN	Auth0 tenant (scheme/slash tolerated)
AUTH0_CLIENT_ID / _SECRET	Auth0 application credentials
DATABASE_URL	SQLite (dev) or Postgres (prod) - <code>postgres://normalized to postgresql://</code>
ADMIN_EMAILS	Comma-separated allowlist for <code>/admin</code> (case-insensitive)
STRIPE_SECRET_KEY	Enables billing
STRIPE_WEBHOOK_SECRET	Verifies webhook signatures
STRIPE_PRICE_PRO / _STUDIO	Stripe price ids per plan

Feature flags derived at startup: `AUTH0_ENABLED` (all three Auth0 vars present), `STRIPE_ENABLED` (`STRIPE_SECRET_KEY` present), `IS_HTTPS` (`APP_BASE_URL` starts with `https://`). Missing keys downgrade the feature, never crash the app.

7 Deployment

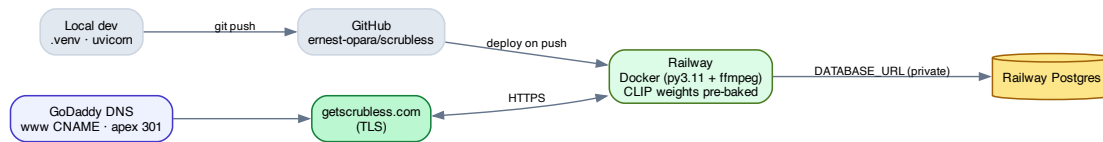


Figure 5: Deployment - push to GitHub triggers a Railway Docker build; the app reads Postgres over the private URL; GoDaddy DNS points the domain at Railway.

- **Container** - Dockerfile on python:3.11-slim, installs ffmpeg, pip-installs requirements.txt, pre-bakes CLIP weights, exposes 8080, runs uvicorn.
- **Host** - Railway (one service) via railway.toml. .dockerignore keeps storage/, .env, .venv/, legacy/ out of the image.
- **Domain** - getscrubless.com via GoDaddy: www CNAME → Railway, apex 301-forwarded, _railway-verify.www TXT. TLS issued by Railway.
- **Durable storage (V2)** - Railway's container filesystem is ephemeral, so the library survives redeploy only with **both**:
 1. a **persistent volume mounted at /app/storage** - holds uploaded videos, extracted frames, and the ChromaDB embeddings; and
 2. **Postgres** via DATABASE_URL = \${Postgres.DATABASE_URL} - holds the users / videos / collections tables.

Without both, uploads vanish on the next deploy. The sample is re-indexed each boot regardless (ingest_sample). On startup restore_state() rebuilds the in-memory caches from the DB and resumes any interrupted indexing.

7.1 Project layout

The backend used to be a single 1.7k-line app.py. It's now 14 sibling modules that import each other in a one-way DAG (config → state → models → embeddings/auth → access/billing/indexing → route modules → app). The frontend is still one index.html (vanilla JS, no build).

```
clipfind/
├─ app.py           # FastAPI entry - middleware, mounts, router wiring, threads
├─ config.py        # env vars, paths, tier caps, enabled flags (cheap; no I/O)
├─ state.py         # in-memory VIDEOS / COLLECTIONS / REELS + SAMPLE_ID
├─ ratelimit.py     # shared slowapi Limiter (imported by every route module)
├─ models.py        # SQLAlchemy + engine + ensure_columns + persist_* + record_event
├─ embeddings.py    # CLIP model + Chroma collection + Claude (describe + summarize)
├─ auth.py          # Auth0 OAuth + current_user + is_admin + /api/auth/* router
├─ access.py        # can_access_* / require_* / require_localhost / safe_video_ext
├─ billing.py       # + gated /storage/{path} router (replaces public static mount)
└─
```

```

|— indexing.py      # ffmpeg + Whisper + process_video + ingest_sample / expiry / restore
|— videos.py       # /api/upload + /api/status + /api/videos/{id} (+ /source)
|— library.py      # /api/library/* + /api/me/library
|— search.py       # /api/search/{id} + /api/qa/{id} + library variants
|— reels.py        # build_reel + /api/reel/*
|— admin.py        # /api/admin/stats + /admin
|— index.html      # entire frontend (vanilla JS + CSS, no build step)
|— requirements.txt
|— Dockerfile, railway.toml, .dockerignore
|— README.md       # one-screen orientation for a fresh visitor
|— assets/         # sample.mp4 (auto-indexed demo) + screenshots used in PITCH
|— scrubby/        # mascot SVGs + favicon (publicly mounted at /scrubby)
|— docs/           # ARCHITECTURE.md (source of truth) + PITCH.md + render.sh + diagrams/
|— storage/        # runtime: uploads, frames, chroma, reels (gitignored)
|— legacy/         # archived earlier Go + React + sidecar stack

```

The **uvicorn app:app** entrypoint is unchanged across the module split, so the Railway deploy needed no configuration update.

8 Security model

Three layers between an attacker and another user's data: ownership gates on every `{id}` route, a gated `/storage` route that replaces the previously public static mount, and rate limits + headers + a prod-secret refusal at boot.

Resource ownership (the “404 not 403” rule). Every `/api/.../{id}` and `/storage/{path}` request flows through `can_access_*` in `access.py`:

- Logged-in user is owner → allowed.
- Resource has no owner AND the caller's session has it in `my_videos / my_collections / my_reels` → allowed (preserves the “no account needed” flow for anonymous uploaders within the same browser session).
- Caller's email is in `ADMIN_EMAILS` → allowed.
- The sample video (`SAMPLE_ID = "sample"`) is world-readable.
- Otherwise: **404** (not 403 - prevents enumeration of valid ids).

Gated `/storage`. `app.mount("/storage", StaticFiles(...))` was removed. `storage_serve` in `access.py` now resolves the path under `STORAGE` (rejecting traversal), inspects the first segment to identify the video / reel, runs the same ownership check, and only then returns `FileResponse`.

Upload extension allow-list. `safe_video_ext()` clamps any user-supplied extension to `VIDEO_EXTS`, defaulting to `.mp4` - uploaders can't land a `.html` or `.svg` on our own origin (closes a stored-XSS via the storage route).

Localhost-only host I/O. `/api/library/scan` and `/api/library/pick` both call `require_localhost()`, so they're usable in self-host but return 404 on a public deploy.

Stripe webhook. Signature-verified via `stripe.Webhook.construct_event` - the only way tier promotions reach the DB.

Prompt-injection hardening. Both Q&A prompts wrap user-controllable text (`<transcript>...</transcript>` `<excerpts>...</excerpts>`, `<question>...</question>`) and instruct Claude to treat anything inside as untrusted data, never as instructions. Best-effort; meaningful while there are no tool-use grants on those endpoints.

Per-IP rate limits. `slowapi Limiter` decorates the sensitive routes - login 20/min, upload 20/h, library-upload 60/h, search 60/min, Q&A 20/min, reel 10/h. In-memory; single-instance assumption.

Boot-time hardening. `app.py` refuses to start on HTTPS if `SESSION_SECRET` is the default or shorter than 32 chars. On HTTPS, session cookies get the Secure flag and the response middleware adds HSTS.

Response headers (every response, by middleware): `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: same-origin`. HTML responses also get a CSP that whitelists Stripe + Auth0 form-actions and blocks framing entirely (`frame-ancestors 'none'`).

9 Admin dashboard

`/admin` serves the SPA, which detects the path and renders the admin view - but only if `is_admin(user)` (the auth state already exposes it on `/api/auth/me`). Non-admins who hit `/admin` directly get bounced back to `/` client-side; the API endpoint backs that with a server-side 403.

`GET /api/admin/stats` returns one JSON blob: users (total + by tier), videos (total + by status + hours indexed), collections, storage used (bytes + GB from an `os.walk` of `STORAGE`), the last 10 signups, and the activity block.

Activity tracking. `record_event(kind)` is called fire-and-forget at the top of every `view / upload / search / qa / reel` handler and inserts one row into the `events` table. The admin endpoint aggregates per kind into `total / d1 / d7` counts plus a 7-element `series` (slot 0 = events from 6–7 days ago, slot 6 = the last 24 h). The SPA renders an inline SVG sparkline per card and a daily-breakdown table whose column labels are derived from `series_starts_at` so timezones stay local to the viewer.

Revenue / MRR / churn live in the Stripe dashboard (linked from the admin page); traffic and logs in Railway. The in-app dashboard deliberately stops at counts.

10 Roadmap - V2 (weekend of 2026-05-23)

Theme: from demo → product. V1 is shipped, deployed, and monetized; V2 makes it durable and sticky.

Foundation:

1. **Durable storage + state - implemented & verified in production.** videos/collections metadata is persisted to Postgres and rebuilt on boot (`restore_state`), and files + embeddings sit on a persistent volume (`/app/storage`). Verified on `getscrubless.com` across a real redeploy: metadata, embeddings, frames, and source files all survived with no re-upload.
2. **Per-user library - implemented.** Signed-in users get a “Your library” panel listing their own videos and collections (GET `/api/me/library`, owner-filtered); click to reopen and re-search. Anonymous uploads still auto-expire (24 h).

Marquee: Library Mode - search across a whole folder. Turn Scrubless from a single-clip tool into a search engine for a video *library*: one query returns the best moments across **every** video in a directory, each result naming its source file + timestamp.

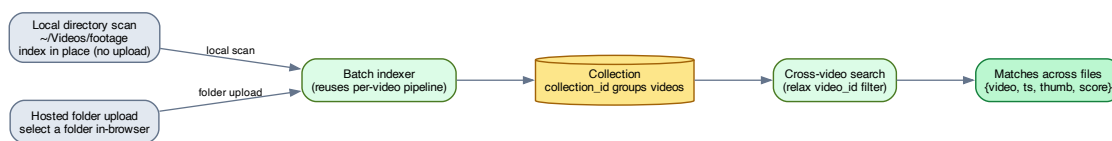


Figure 6: V2 Library Mode - both ingestion modes (local scan first, hosted upload second) feed one batch indexer; a collection scopes cross-video search.

- **Cross-video search.** Frames already carry `video_id` in ChromaDB, so this is mostly relaxing the per-video where filter and carrying the source video into each result. The expensive part (per-video indexing) already exists.
- **Collections.** A `collection_id` grouping scopes a query to “this folder.”
- **Two ingestion modes - both committed, local first:**
 - *Local directory scan* (self-hosted) - **implemented** (`v2-library-mode` branch): POST `/api/library/scan` walks a folder, indexes each video in place (nothing uploaded); the UI shows live per-video progress and searches across all of them. Frontend folder panel appears only on localhost.
 - *Hosted folder upload* - **implemented**: select a folder in the browser (`webkitdirectory`); the client uploads each video into a new collection (POST `/api/library/create` then `/api/library/{id}/upload`) and searches across them. Reuses the same cross-video engine. Per-file tier caps apply.
- **Constraint.** Indexing is CPU-bound (CLIP on CPU); first index of a large folder takes time - durable state + a progress UI make that acceptable.

Status: both ingestion modes are built and verified, and merged to main / deployed. Local scan = in-place (no upload, localhost only); hosted upload works anywhere. Caveat: uploaded collections live on Railway’s ephemeral disk, so they don’t survive a redeploy yet - the durable-storage foundation closes that gap.

Deferred: clip export + share links, YouTube ingest (legal), search-quality (hybrid ranking), React rewrite, face recognition (legal review), native mobile, GPU inference.

11 Roadmap - V3

V2 is shipped (Library Mode - both ingestion modes, durable storage, per-user library). V3 moves up the value chain - from *finding* moments to *answering* and *sharing*. Chosen arc, in order:

1. **Auto-chapters + summary - in progress.** On index, a Claude call over the transcript (summarize_video) produces a 2–3 sentence summary and 4–8 timestamped chapters, stored on the videos row and shown under the player (click a chapter to seek). Best-effort: needs a transcript + Claude key.
2. **Clip export + highlight reels - implemented.** “Make highlight reel” stitches the top search-result moments (single video or across a library) into one 720p MP4 via ffmpeg - async build (POST /api/reel → GET /api/reel/{id} poll) with download + a shareable link. A one-moment reel is effectively a clip export. Video-only for now; audio is a future add.
3. **Conversational video Q&A - implemented (single video).** A Search/Ask tab in the workspace: ask a question and get a concise answer grounded in the transcript with inline **[Ns] citations** you click to seek the player (POST /api/qa/{id} rebuilds a timestamped transcript from the segments and asks Claude). Library-wide Q&A is a natural future extension.

V3 arc complete - chapters, reels, and Q&A all shipped.

Considered, lower priority: search-by-image, YouTube + cloud connectors, public API + browser extension, team workspaces, infra scale (queue / GPU / S3), face/object recognition (legal risk).

12 Known limitations (V1)

- **5-second frame granularity.** Moments shorter than the interval can be missed.
- **Single-process / single-node.** No horizontal scaling; CLIP runs on CPU.
- **Storage is one local volume.** Durable across restarts (DB + persistent volume), but not object storage / multi-node - fine for a single Railway service.
- **No clip export, no YouTube ingest, no face recognition** (deferred by spec).

13 About this document

The `.md` is the editable **source** (plain text - easy to diff, version, and edit precisely). The `.pdf` is the **rendered output** you read and share. Edit the Markdown (or a `diagrams/*.dot`), run `./docs/render.sh`, and `pandoc + xelatex` rebuild the PDF with freshly rendered Graphviz diagrams.

It is kept current as the project evolves (not mechanically every prompt) and pushed to GitHub whenever it changes. The change log records each meaningful update.

14 Change log

Date	Trigger	Change to this doc
2026-05-22	Initial request: PDF on the architecture.	Created the architecture document and rendered it to PDF.
2026-05-22	Auth0 tenant friendly-name fix.	No architecture change (cosmetic Auth0 dashboard setting).
2026-05-22	Auth0 dev-keys alert cleared.	No architecture change (own Google OAuth keys in Auth0).
2026-05-22	“what is v2 for this weekend”	Added the V2 roadmap (foundation + marquee).
2026-05-22	“use scrubless on a directory full of videos”	Set V2 marquee to Library Mode (cross-video search + batch ingestion).
2026-05-22	“do both, local first; commit v1 totally”	Locked both ingestion modes (local first); committed docs/, tagged v1.0.
2026-05-22	“fix structure; make it diagrammatic; push on update”	Restructured the document; added six Graphviz vector diagrams; switched cadence to “on meaningful change” + auto-push; documented the <code>.md/.pdf</code> split.
2026-05-22	“proceed” (build V2)	Implemented Library Mode - local directory scan + cross-video search (backend + UI) on <code>v2-library-mode</code> . Added the new endpoints to the API table, <code>collection_id/COLLECTIONS</code> to the data model + diagram, and marked local scan implemented in the roadmap.
2026-05-22	“isn’t it better to select the folder in Finder?”	Added a native folder picker (<code>POST /api/library/pick</code> via <code>osascript</code>) + a Browse button, since browsers can’t expose a folder’s absolute path to JS. Text-path input kept as a fallback.

Date	Trigger	Change to this doc
2026-05-22	"1 then 2" (merge+deploy, then phase 2)	Merged Library Mode to main (deployed). Built phase 2 - hosted folder upload (/api/library/create + /api/library/{id}/upload, webkitdirectory UI); both ingestion modes now implemented. Updated the V2 diagram + roadmap.
2026-05-22	"yes tackle it now" (durable storage)	Durable storage foundation: persisted videos/collections to the DB + restore_state() on boot (resumes interrupted indexing; idempotent re-index). Library now survives restart - verified locally. Requires a Railway volume at /app/storage + DATABASE_URL. Rewrote the data-model diagram; updated deployment + limitations + roadmap.
2026-05-22	"verify persistence"	Verified durable storage on production across a real redeploy: collection metadata (Postgres), embeddings + frames + source files (volume) all survived with no re-upload; delete path confirmed on prod. Volume + DATABASE_URL are correctly configured.
2026-05-22	"do the ui" (per-user library)	Added GET /api/me/library (owner-filtered videos + grouped collections) and a "Your library" panel so signed-in users reopen/re-search past videos and folders; header logo links home. Completes the V2 per-user library item.
2026-05-22	"10 ideas, ranked" → "yes do that" (V3 start)	Locked V3 arc (auto-chapters → clip export → Q&A). Built #1 auto-chapters + summary: summarize_video() (Claude over transcript) at index time, persisted via new videos.summary/chapters columns + ensure_columns() migration, exposed on /api/status, rendered as a clickable chapter list + summary under the player. Added the V3 roadmap section + data-model updates.

Date	Trigger	Change to this doc
2026-05-22	“highlight reels” (V3 #2)	Built highlight reels : POST <code>/api/reel</code> (async ffmpeg concat of trimmed, uniformly-scaled 720p moments) + GET <code>/api/reel/{id}</code> poll; “Make highlight reel” button in both workspaces → modal with player, download, and shareable link. Verified locally (3×4s → 12.0s 1280×720 MP4). Video-only for v1.
2026-05-22	“proceed” (V3 #3 Q&A)	Built conversational video Q&A : POST <code>/api/qa/{id}</code> rebuilds a timestamped transcript from the segments and asks Claude for a concise, citation-grounded answer; the workspace gains a Search/Ask tab that renders clickable [Ns] citations seeking the player. Single-video, transcript-grounded. Completes the V3 arc (chapters + reels + Q&A).
2026-05-22	UI feedback from prod screenshots	Fixed three UX issues: (1) workspace grid <code>min-width: 0 + minmax(0, ...)</code> so search results can’t squeeze the player; (2) folder upload/index now shows a progress bar + completion banner ; (3) added a “ My library ” header button (signed-in) to reach the per-user library from any view.
2026-05-22	“so much space... utilise it better”	Widened the workspace (1040→1400px) while keeping the landing focused (760px), and made the player sticky with results flowing down the page - uses the horizontal + vertical space and keeps the video visible while scrolling results.
2026-05-22	“lots of space under the player - what to add?”	Filled the library player’s left column: now-playing summary + clickable chapters under the player + a “ Videos in this folder ” list (jump to any video without searching). Reuses <code>/api/status</code> + <code>/api/library/{id}</code> ; no backend change.

Date	Trigger	Change to this doc
2026-05-23	"Add the Search/Ask tab to the library"	Added cross-folder Q&A : <code>POST /api/library/{id}/qa</code> retrieves the top moments across the collection and asks Claude for a cited answer; the library workspace gains Search/Ask tabs, and answer citations are clickable chips (" <code>[file @ 0:15]</code> ") that load that video and seek to the moment.
2026-05-23	"can you create a pitchdeck"	Added a YC/seed pitch deck (<code>docs/PITCH.md</code> → <code>PITCH.pdf</code>): 12 beamer/metropolis slides on the app's coral+charcoal brand, with a new <code>diagrams/pitch-flow.dot</code> product-flow figure. No system change; <code>render.sh</code> now builds both the architecture doc and the deck.
2026-05-23	"compare against ElevenLabs... close the gaps"	Hardened the deck against the ElevenLabs pre-seed benchmark (14 slides): added a live-demo slide (real CDP-captured screenshot of search on <code>getscrubless.com</code> , stored in <code>docs/assets/</code>), a quantified-pain callout, a nested TAM/SAM/SOM/beachhead market figure (<code>diagrams/market.dot</code>) + near-term revenue, a 2x2 competition matrix (<code>diagrams/competition.dot</code>) replacing the table, and a "why we win" moat slide.
2026-05-23	"can you make this a powerpoint?"	Added an editable PowerPoint build (<code>md2pptx.py</code> → <code>pandoc</code>): rewrites the beamer-only LaTeX (<code>\alert</code> , <code>\vspace</code> , <code>\begin{center}</code> , size macros) into plain Markdown and folds image captions onto their slides. Single canonical source (<code>PITCH.md</code>).
2026-05-23	"make it match"	Made the PowerPoint brand-match the PDF : <code>PITCH.pptx</code> is now built by <code>pdf2pptx.py</code> (python-pptx) - each <code>PITCH.pdf</code> page becomes a full-bleed 16:9 slide image, so it's pixel-identical to the metropolis deck and immune to font substitution. The editable variant lives on as <code>PITCH-editable.pptx</code> .

Date	Trigger	Change to this doc
2026-06-08	“/admin operator dashboard”	Added /admin (gated by ADMIN_EMAILS) + /api/admin/stats: users-by-tier, videos-by-status + hours indexed, collections, storage size, last 10 signups. SPA renders stat cards + a recent-signups list with a link-out to Stripe/Railway for revenue and traffic.
2026-06-08	“site views + relevant stats + sparkline + daily breakdown”	Added an events(kind, at) table written fire-and-forget by record_event() from every view / upload / search / qa / reel handler; admin stats now include total + 24 h + 7 d counts and a 7-element series per kind, rendered as an inline SVG sparkline on each card and a daily breakdown table.
2026-06-08	“security audit (50 vuln list) + fixes”	Two-commit hardening pass: replaced the public /storage static mount with the gated storage_serve; added can_access_* / require_* helpers and applied them to every {id}/{cid} route (404, not 403, to prevent enumeration); per-session allowlist (remember_anon_resource) for anonymous uploaders; safe_video_ext extension allow-list closes stored-XSS; Collection.owner added (migration via ensure_columns); reels carry owner and check moment access at create time; /api/library/scan + /pick now require_localhost; slowapi rate limits on login / upload / search / qa / reel; prompt-injection fences on both Q&A prompts; app.py refuses to boot on HTTPS with a default / <32-char SESSION_SECRET; Secure cookies + HSTS on HTTPS; security-header middleware (nosniff, X-Frame-Options: DENY, Referrer-Policy, CSP on HTML).

Date	Trigger	Change to this doc
2026-06-08	"split app.py into modules"	Refactored the 1.7k-line <code>app.py</code> into 14 sibling modules in a one-way DAG (<code>config</code> → <code>state</code> → <code>models</code> → <code>embeddings/auth</code> → <code>access/billing/indexing</code> → <code>route modules</code> → <code>app</code>). <code>app.py</code> shrinks to ~100 lines and just wires middleware, mounts, routers, and the background threads. Zero behavior change; <code>uvicorn app:app</code> entrypoint preserved.
2026-06-08	"doc improvements"	Refreshed this document for the module split + everything shipped since 05-23: rewrote Project layout as the 14-module map, added <code>Security model</code> and <code>Admin dashboard</code> sections, updated the API table (<code>/admin</code> , <code>/api/admin/stats</code> , <code>gated /storage</code> , <code>rate-limit annotations</code>), added <code>ADMIN_EMAILS</code> to env vars, added the <code>events</code> table to the data model, refreshed the <code>tech-stack</code> table (<code>slowapi</code> , <code>python-pptx</code>). Also wrote a fresh top-level <code>README.md</code> .